Query Refinement for Radius-Bounded k-Core Queries

Zefang Dong, Chuanyu Zong, Xiaochun Yang, *Senior Member, IEEE*, Bin Wang, Boce Chu, Yaqi Wang, and Huaijie Zhu

***Abstract*—Radius-bounded k-core queries (RB-k-core queries) in geo-social networks aim to identify all k-cores containing a given query vertex q, where all vertices in each k-core fall within a circle defined by a specified query radius r. These queries are widely used in applications such as team formation and event organization. However, specifying query parameters k and r can be challenging for users without domain expertise, often resulting in misaligned query results. Specifically, some expected vertices may be missing, while unexpected vertices may appear in the results. To address this issue, we investigate the problem of *exploring optimal* refined *parameters* for *expected* (EOPE) and *unexpected* (EOPU) results in RB-k-core queries. The goal is to explore optimal parameters that ensure the expected vertex ω (or unexpected vertex ψ) and query vertex q appear (or do not appear) in the same RB-k-core. For the EOPE problem, we first propose two baseline algorithms: *PriorityR* and *HybridR*. To improve efficiency, we develop two more advanced algorithms: *PriorityK* and *HybridK*. Additionally, we introduce a novel index called *HCR-Tree*, based on hierarchical coreness of vertices and R-Tree, to enhance exploration efficiency. For the EOPU problem, we begin with a basic solution (*BS*) and then design the segmentation algorithm *SA*, which incorporates effective pruning and termination strategies. We conduct extensive experiments on five real-world geo-social network datasets. The results demonstrate that our proposed algorithms effectively explore optimal parameters. Among them, *HybridK* proves most effective for EOPE, while *SA* performs best for EOPU. Furthermore, *HCR-Tree* outperforms R-Tree for both EOPE and EOPU problems.**

***Index Terms*—Geo-social networks, RB-k-core queries, expected results, unexpected results, optimal parameters.**

Received 4 February 2025; revised 7 March 2026; accepted 2 June 2026. Date of publication 10 June 2026; date of current version 8 August 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFF0617702, in part by the National Natural Science Foundation of China under Grant U22A2025, Grant 62232007, and Grant U23A20309, in part by the Key R&D Program of Liaoning under Grant 2025JH2/101800116, and in part by 111 Project under Grant B16009. Recommended for acceptance by G. Wang. *(Corresponding authors: Chuanyu Zong; Xiaochun Yang.)*

Zefang Dong, Xiaochun Yang, Bin Wang, and Yaqi Wang are with Northeastern University, Shenyang 110819, China (e-mail: dongzefang@stumail.neu.edu.cn; binwang@mail.neu.edu.cn; yangxc@mail.neu.edu.cn; wangyq@stumail.neu.edu.cn).

Chuanyu Zong is with Shenyang Aerospace University, Shenyang 110136, China (e-mail: zongcy@sau.edu.cn).

Boce Chu is with the 54th Research Institute of China Electronics Technology Group Corporation, Shijiazhuang 050081, China (e-mail: Bocc012628077@163.com).

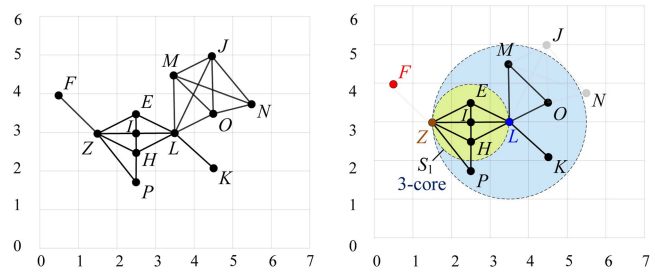
Huaijie Zhu is with Sun Yat-sen University, Guangzhou 519082, China (e-mail: zhuhuaijie@mail.sysu.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3702049

I. Introduction

In recent years, wireless communication technologies and mobile devices have become ubiquitous, leading to the emergence of numerous geo-social networks such as Facebook, Foursquare, and Twitter. These networks integrate social connections with users' geo-spatial information [1], bridging the gap between physical and virtual social interactions. This integration enables new applications for group-based activity planning and scheduling. For example, when organizing board games, a Facebook user may want to gather a group of nearby individuals (within a circle of radius r) where each participant has at least k friends in the group. To address this need, the RB-k-core query problem has been investigated to efficiently identify socially cohesive user groups with spatial constraints [2], [3]. Formally, given a geo-social network $G(V, E)$, a query vertex $q \in V(G)$, a query radius r, and a positive integer k, the RB-k-core query finds all k-cores containing q where all vertices in each k-core lie within a circle of radius r centered at q. Fig. 1 illustrates an RB-k-core query example, where vertices represent users, edges denote friendships, and each vertex v has a location in two-dimensional space. For query vertex L with parameters $k = 3$ and $r = 1$, the query returns $S_1 = \{L, E, Z, H, I\}$, as shown in Fig. 1(b). RB-k-core queries have numerous practical applications. For example, Facebook's Event-For-You service effectively recommends events to users based on their social relationships and geographic locations.

Unfortunately, query results do not always align with user needs. For example, when using RB-k-core queries for activity recommendations, the system may recommend activities organized by entities the user dislikes due to shared friends or proximity. Conversely, activities organized by close friends might be excluded due to excessive distance or weak social connections, particularly for new users. Similarly, when searching for high-impact "opinion leaders" in a specific region, the system may overlook users who exert significant influence within specific circles but have few followers, or include "zombie accounts" with many followers but low interaction quality. In other words, some expected vertices may be missing, while unexpected ones may appear in the final results. Therefore, we aim to develop a method to explain these discrepancies and adjust queries to better meet user needs. In RB-k-core queries, parameter k controls result cohesion, while r determines spatial proximity. However, users without domain expertise often struggle to specify appropriate values for k and r, requiring multiple query revisions to



(a) Geo-social network (b) RB-k-core

Fig. 1. An example of EOPE (EOPU) problem.

obtain desirable results. For example, in Fig. 1, for query vertex (user) L, an RB-k-core S_1 is obtained with parameters $k = 3$ and $r = 1$. User L may question why vertex F is excluded or why vertex Z is included. This leads to why-not and why questions regarding the inclusion of specific vertices. The goal is to explore optimal refined parameters that would include expected vertices like F while excluding unexpected ones like Z. It should be noted that including expected vertices may introduce new ones, while excluding unexpected vertices may remove related ones. Revisiting the activity recommendation scenario: relaxing the cohesion constraint to include close friends may also introduce mutual friends. Conversely, tightening the constraint to exclude disliked organizers may remove vertices with weak connections to the user. We believe this chain effect can represent user preferences to some extent. However, to prevent this effect from being overly amplified, our framework's optimal refinement parameters aim to minimize the difference between the refined and initial query results. In the conference version of our work [4], the proposed framework effectively addressed the question of why expected vertices are missing. Extending this capability to handle unnecessary vertices aims to establish a complete and practical RB-k-core framework. By addressing gaps in real-world application requirements, this extension confronts the unique and more complex technical challenges through the design of novel efficient algorithms. This expanded work elevates the framework from answering "why not" to addressing both "why not" and "why" at a higher conceptual level, thereby enhancing the utility of the research.

Although many efforts have been made to answer why-not (or why) questions by exploring optimal refined parameters based on query refining model [5], [6], [7], [8], [9], [10], such as exploring optimal query sentences for SQL queries [5]; exploring optimal query parameters for spatial keyword top-k queries [6], structural graph clustering [7], and RB-k-core searches [8]; exploring optimal query graphs for similar graph matching [9] and subgraph queries [10], and so on. However, the techniques for why-not questions on RB-k-core queries in [8] only refine the parameter k or r, they cannot address all the missing cases of why-not questions on RB-k-core queries. For example, as shown in Fig. 1(b), if the expected vertex is F, F cannot appear in the same RB-k-core with L by only refining the parameter k or r. That is to say, only by refining the two parameters k and r simultaneously could F appear in the same RB-k-core with L. For example, when the original parameters are refined as $k' = 1$ and $r' = 1.58$, one RB-k'-core $S' = \{L, H, Z, E, P, F, I\}$ is obtained that contains both the query vertex L and the expected vertex F. Moreover, to minimize the size of results for refined queries, the quality of refined RB-k-core queries by refining the two parameters simultaneously is better than that by only refining one parameter. In sum, all the existing explaining techniques for why-not questions cannot be directly used for efficiently exploring the optimal query parameters simultaneously for expected results on the RB-k-cores queries. At the same time, there is no technique available to explain why questions on RB-k-cores queries. Thus, we investigate the problem of exploring optimal refined parameters simultaneously for expected (or unexpected) results on RB-k-core queries, called *EOPE* (or *EOPU*).

Given a query vertex q, an RB-k-core query $Q(k, r)$, and an expected vertex ω (or unexpected vertex ψ), the EOPE problem aims to find optimal refined parameters that ensure both ω and q appear in the same RB-k-core. Conversely, the EOPU problem seeks parameters that guarantee ψ and q do not appear in the same RB-k-core. Fig. 1(b) illustrates an example. Consider an RB-k-core query $Q(3, 1)$ for query vertex L, with expected vertex F and unexpected vertex Z. By simultaneously refining parameters k and r, we obtain optimal parameters $\{k' = 1, r' = 1.58\}$ for EOPE and $\{k' = 3, r' = 0.999\}$ for EOPU. For the EOPE problem, parameters $\{k' = 1, r' = 1.58\}$ yield an RB-k-core containing both L and F with minimal size. For the EOPU problem, parameters $\{k' = 3, r' = 0.999\}$ ensure no RB-k-core contains both L and Z, while maintaining maximal result size.

In the conference version of our work [4], we presented preliminary research on exploring optimal refined parameters, specifically focusing on the EOPE problem. The associated challenges and solutions are discussed below.

For exploring optimal refined queries for why-not questions, two fundamental principles must be satisfied: (i) the original query results should be retained, and (ii) the number of irrelevant vertices should be minimized. *The primary challenge in addressing the EOPE problem is to efficiently explore optimal refined parameters that simultaneously include expected vertices in query results while minimizing the inclusion of irrelevant vertices.*

To solve the EOPE problem, we first introduce a unified explanation framework for why-not questions on RB-k-core queries. A naive approach to simultaneously tuning r and k would involve evaluating all candidate parameter pairs. However, verifying numerous parameter pairs using the most efficient RB-k-core search algorithm, RotC+ [3], proves inefficient and impractical. To reduce the number of unqualified parameter pairs, we propose two baseline exploration algorithms: *PriorityR* and *HybridR*, based on effective bounds of refined r' and a dichotomous strategy, respectively. To further improve efficiency in exploring optimal parameters for expected results, we develop two advanced algorithms: *PriorityK* and *HybridK*, which utilize the effective bound of refined k' and continuous convergence bounds of both refined k' and r'. Additionally, we introduce a

novel index called *HCR-Tree*, integrating hierarchical coreness of vertices with R-Tree [11], to accelerate these algorithms.

To expand upon our previous work, we now describe the challenges of the EOPU problem and our proposed solutions.

For exploring optimal refined queries for why questions, two key principles must be maintained: (i) the original query results should be preserved as much as possible, and (ii) irrelevant vertices should not be introduced. *The primary challenge in solving the EOPU problem is to efficiently obtain optimal refined parameters while minimizing the exclusion of initial query results.*

To address the EOPU problem through simultaneous refinement of parameters k and r, we first present an explanation framework for why questions on RB-k-core queries. A naive approach would apply the RotC+ algorithm to evaluate all possible parameter pairs and select the one that maximizes the retention of initial query results. However, this method is computationally prohibitive due to its high cost. As an initial solution, we introduce a basic approach *BS* based on the three-vertex method, which reduces the number of candidate parameter pairs to a constant. Subsequently, we design an enhanced solution *SA* that incorporates a segmentation strategy and multiple efficient pruning techniques. Although RotC+ was previously employed to identify optimal parameter pairs, its primary focus on subgraph search makes it suboptimal for our objective of maximizing vertex sizes in query results. Therefore, we design algorithm *FA* to replace RotC+, providing a more efficient and targeted approach for identifying optimal refined parameter pairs.

The main contributions of this paper are as follows:

- We propose a unified explanation framework for why-not and why questions on RB-k-core queries, which enables simultaneous exploration of optimal refined parameters for both expected and unexpected vertices.
- To address the EOPE problem [4], we first introduce two baseline algorithms, *PriorityR* and *HybridR*, to ensure the inclusion of expected vertices in refined RB-k'-core query results. Subsequently, we propose two advanced algorithms, *PriorityK* and *HybridK*, which significantly improve the efficiency of identifying optimal parameters.
- For address why questions on RB-k-core queries, we first present a basic solution *BS* to exclude unexpected vertices from refined RB-k'-core results. Building upon this foundation and incorporating pruning and termination strategies, we design the segmentation algorithm *SA* to efficiently identify optimal refined parameter pairs for unexpected results.
- We conduct comprehensive experiments on five real-world geo-social networks to evaluate our parameter exploration algorithms. The experimental results demonstrate that our algorithms effectively generate high-quality refined parameters for both expected and unexpected results in RB-k-core queries.

The remainder of this paper is organized as follows. Section II introduces fundamental concepts and problem definitions. Section III presents the explanation framework for expected and unexpected vertices in RB-k-core queries. Section IV reviews our proposed algorithms for solving the EOPE problem.

TABLE I
SUMMARY OF NOTATIONS

Symbol	Description
$G(V, E)$	A geo-social graph
$N_G(v)$	The neighbors of vertex v in G
$d(v, u)$	The euclidean distance between vertices v and u
$\mathcal{C}(v, r)$	A circle centered at vertex v with radius r
$\mathcal{C}_B(v, u)$	A set of binary-vertex-bounded circles with boundary vertices u and v
$cnes_G[v]$	The coreness of vertex v in G
$core_k(v)$	A connected k-core containing a vertex v
$core_k(q, \omega)$	A connected k-core containing both query vertex q and expected vertex ω.
$deg_G(v)$	The degree of vertex v in G
G_k^r, $\mathcal{B}$	A RB-k-core in G
$V(\mathcal{C})$	The set of vertices within the circle $\mathcal{C}$
$G(\mathcal{H})$	A subgraph in G generated by the vertices in $\mathcal{H}$

Section V describes our basic solution BS and segmentation algorithm SA, which simultaneously refine both parameters for the EOPU problem. Section VI presents the experimental results and analysis. Section VII reviews related work, and Section VIII provides concluding remarks.

II. Problem Statement

Given a geo-social graph $G(V, E)$, where V denotes the set of vertices in G and E denotes the set of edges in G. $N_G(v)$ denotes the set of neighbors of a vertex v in G, which is formalized as $N_G(v) = \{u | u \in V, (u, v) \in E\}$. Each vertex v has a location denoting its position in a two-dimensional space. Table I summarizes the notations used in the paper.

To define the *RB-k-Core Query*, we first introduce the definitions of *k-Core* and *Minimum Covering Circle*.

Definition 1. k-Core [12]: Given a geo-social graph G and an integer k, the k-core of graph G is the maximal subgraph of G, which is denoted as H_k, such that $\forall v \in H_k, deg_{H_k}(v) \geq k$.

Definition 2. Coreness [12]: Given a geo-social graph G and a vertex $v \in G$, the coreness of v is the highest order of a k-core that contains v, and it is formalized as $cnes_G[v]$.

Definition 3. Minimum Covering Circle (MCC for short) [3]: Let $\mathcal{H}$ be a set of vertices, the circle that encloses all the vertices in $\mathcal{H}$ with the smallest radius is defined as the minimum covering circle of $\mathcal{H}$. The vertices that lie on the boundary of a MCC are called $boundary$ vertices.

Definition 4. RB-k-Core Query [3]: Given a geo-social graph G, a query vertex q, an integer k, and a query radius r, the RB-k-core query is to find all RB-k-cores in G, and each RB-k-core G_k^r should satisfy the following constraints: 1)**Connectivity** $constraint$. G_k^r is connected and q belongs to G_k^r; 2) **Social** $constraint$. For any vertex $v \in G_k^r$, the degree of v is not less than k; 3) **Spatial** $constraint$. The MCC of G_k^r has a radius $r' \leq r$; 4) **Maximality** $constraint$. There exists no other RB-k-core $G_k'^r \supseteq G_k^r$ satisfying 1), 2), and 3).

Definition 5. EOPE Problem [4]: Given a geo-social graph $G(V, E)$, a query vertex q, a RB-k-core query $Q(k, r)$, the result R_Q consists of l RB-k-cores $\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_l$, and an expected vertex ω, the problem of EOPE is to find the optimal refined query $Q'(k', r')$ that satisfies the following **conditions**: 1) $k' \leq$

$k, r' \geq r$; 2) $\exists \mathcal{B}'_i \in R_{Q'}$, $\mathcal{B}'_i$ is a $core_{k'}(q, \omega)$; 3) There exists no other refined query Q'' satisfying the above two conditions, and $|\bigcup_{i=1}^{n} \mathcal{B}''_i| < |\bigcup_{i=1}^{n} \mathcal{B}'_i|$, $\mathcal{B}'_i \in R_{Q'}$, $\mathcal{B}''_i \in R_{Q''}$.

Definition 6. EOPU Problem: Given a geo-social graph $G(V, E)$, a query vertex q, a RB-k-core query $Q(k, r)$, the result R_Q consists of l RB-k-cores $\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_l$, and an unexpected vertex ψ, the problem of EOPU is to find the optimal refined query $Q'(k', r')$ that satisfies the following three **conditions**: 1) $k' \geq k$, $r' \leq r$; 2) $\forall \mathcal{B}'_i \in R_{Q'}$, $\nexists core_{k'}(q, \psi) \in \mathcal{B}'_i$; 3) There exists no other refined query Q'' satisfying the above two conditions, and $|\bigcup_{i=1}^{n} \mathcal{B}''_i| > |\bigcup_{i=1}^{n} \mathcal{B}'_i|$, $\mathcal{B}'_i \in R_{Q'}$, $\mathcal{B}''_i \in R_{Q''}$.

Condition 1) constrains the bounds of the refined parameters k' and r'. Condition 2) ensures that ψ cannot be contained in any RB-k-cores, while noting that the query vertex q must be retained. Condition 3) ensures that the original query results are preserved to the greatest extent possible.

III. Explanation Framework

In this section, we first analyze the missing cases and unnecessary cases on RB-k-core queries.

Missing cases: The reason for missing expected vertices is the inability to find a connected k-core containing both query vertex q and expected vertex ω in a circle with radius r. It contains five cases as follows.

1) $d(q, \omega) \leq 2 \cdot r$, $cnes_G[\omega] \geq k$, $cnes_{G(C(q,2\cdot r))}[\omega] \geq k$, $\forall core_k(q, \omega) \in G(\mathcal{C}(q, 2 \cdot r))$, the radius of MCC of $core_k(q, \omega))$ is greater than r;
2) $d(q, \omega) \leq 2 \cdot r$, $cnes_G[\omega] \geq k$, $cnes_{G(\mathcal{C}(q,2\cdot r))}[\omega] < k$;
3) $d(q, \omega) \leq 2 \cdot r$, $cnes_G[\omega] < k$, $cnes_{G(\mathcal{C}(q,2\cdot r))}[\omega] < k$;
4) $d(q, \omega) > 2 \cdot r$, $cnes_G[\omega] \geq k$;
5) $d(q, \omega) > 2 \cdot r$, $cnes_G[\omega] < k$.

Based on the rationale of RB-k-core queries, refining single parameter k can only address the missing cases 1), 2), and 3), refining single parameter r can only address the missing cases 1), 2), and 4). Moreover, given two refined parameter pairs $\{k, r'\}$ and $\{k', r\}$, $\exists k'' \in [k', k]$, $r'' \in [r, r']$, the refined quality of $\{k'', r''\}$ is better than or equal to that of both $\{k, r'\}$ and $\{k', r\}$. Refining two parameters simultaneously can address all the missing cases and we explore the optimal two refined parameters to improve the quality of refinement. These illustrate the superiority of our framework.

Unnecessary Case: For the EOPU problem, the unexpected vertex ψ always satisfies $d(q, \psi) \leq 2 \cdot r$, $cnes_{G(\mathcal{C}(q,2\cdot r))}[\psi] \geq k$, and $\exists core_k(q, \psi) \in G(\mathcal{C}(q, 2 \cdot r))$. Furthermore, it is evident that one or more RB-k-cores must exist within $\mathcal{C}(q, 2 \cdot r)$, where ψ and q exhibit varying distances and coreness values. These properties facilitate the determination of refined queries. However, obtaining these properties requires multiple rounds of RB-k-core queries, indicating that the EOPU problem is more complex than the EOPE problem. We focus exclusively on the simultaneous refinement of both parameters for the EOPU problem, based on the following rationales: First, similar to the EOPE problem, simultaneously refining both parameters may result in less loss of initial query results compared to refining only one parameter, thereby achieving higher refinement quality. Second, the single parameter refinement method lacks interpretability for all unnecessary vertices. For example, if $\forall G(C(q, 2 \cdot r))^r_k$, where $Cnes_{G(C(q,2\cdot r))^r_k}[q] = Cnes_{G(C(q,2\cdot r))^r_k}[\psi]$, satisfying $Cnes_{G(C(q,2\cdot r))^r_k - \psi}[q] < k$, that is, removing vertex ψ necessarily causes the coreness of vertex q to drop below k. In this case, a solution cannot be obtained through the single parameter refinement method because if ψ is removed, q must also be removed, making the query unsolvable.

Explanation Framework: To address all missing and unnecessary cases while achieving optimal refinement quality, we propose an explanation framework for expected and unexpected vertices in RB-k-core queries, as illustrated in Fig. 2. Both the EOPE and EOPU problems can be solved using naive approaches, and despite differences in the actual values of k and r, effective bounds can be established for both problems. However, given that the two problems have opposing requirements for query results—one requiring the inclusion of expected vertices and the other requiring the exclusion of unexpected vertices—we design tailored explanation approaches for each problem separately.

1) EOPE Problem Solution: For an input EOPE problem, a ***naive approach*** involves verifying all possible candidate refined parameter pairs until obtaining the optimal pair, as shown in the top-middle part of Fig. 2. However, this approach requires verifying numerous refined parameter pairs by invoking the RotC+ algorithm, which is computationally prohibitive. The RotC+ algorithm is currently the best-known method for RB-k-core queries. It takes the query vertex q and parameters k and r as input and outputs the vertex set that satisfies both cohesion and distance constraints and includes the query vertex q. By replacing the input parameters with candidate refined parameter pairs $\{k', r'\}$, we obtain the corresponding results, from which we identify the optimal parameter pair that minimizes modifications to the original results while including the expected vertex ω. Although effective bounds can be established based on q and ω to prune non-optimal parameter pairs, the number of remaining pairs remains substantial. To address this challenge, we explore two ***baseline algorithms*** and two ***effective algorithms***, as depicted in the bottom-middle part of Fig. 2. The candidate pairs identified by our algorithms are then validated by invoking the RotC+ algorithm. Finally, the optimal parameters k'_{opt} and r'_{opt} are returned, yielding refined query results with minimal size that contain the expected vertex ω.

2) EOPU Problem Solution: For an input EOPU problem, a naive approach would execute the RotC+ algorithm for all possible candidate parameter pairs, which is computationally infeasible. To overcome this limitation, we first design a ***basic solution***, ***BS***, which reduces the number of explored parameter pairs to a constant. Building upon this foundation, we develop effective pruning and termination strategies, culminating in an ***effective solution***, ***SA***. In our previous work, RotC+ was employed to validate the explored pairs. To further accelerate this process, we design the fast parameter validation method ***FA***, which efficiently identifies the optimal parameters.

Discussion on Multiple Vertices: Considering practical application requirements, we discuss the multiple vertices scenario in which both expected and unexpected vertices coexist. We introduce the cases as follows:

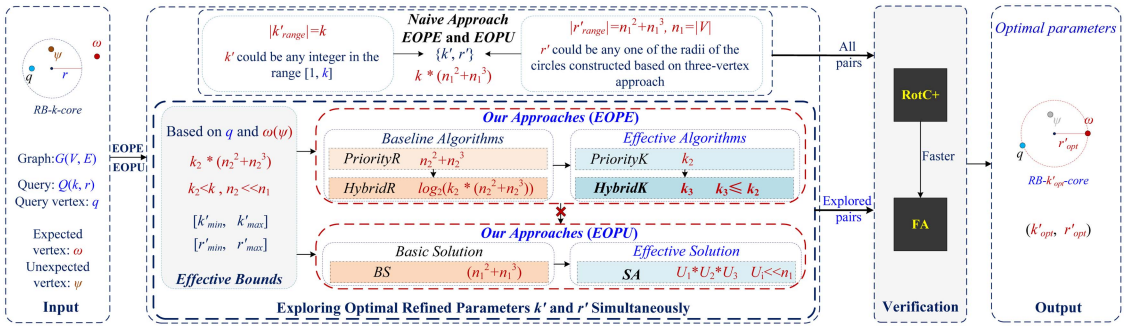


Fig. 2. Explanation framework.

Given a query vertex q, an expected vertex ω, and an unexpected vertex ψ, where $\exists core_{k_i}(\omega)$ and $core_{k_j}(\psi)$ with maximal k_i and k_j within G, we consider two cases: (1) $k_i > k_j$ or $d(q,\omega) < d(q,\psi)$; (2) $k_i \leq k_j$ and $d(q,\omega) \geq d(q,\psi)$. For case (2), we define the parameter set $P_r = \{\{k_x, r_x\} \mid x \in [0,n]\}$ such that for all $\{k_x, r_x\} \in P_r$, there exists $core_{k_x}(q,\omega)$ within $C(q, 2\cdot r_x)$, and the radius r'_x of the minimum covering circle (MCC) of $core_{k_x}(q,\omega)$ satisfies $r'_x \leq r_x$. Additionally, no parameter pair $\{k_y, r_y\} \notin P_r$ satisfies these conditions.

In case (1), the current framework can handle multiple vertices scenarios. The simplest approach is to either increase the refined k' to k_i or decrease the refined r' to $d(q,\omega)/2$, which ensures the inclusion of ω while excluding ψ, though this solution may not be optimal. To obtain the optimal refinement and determine whether a solution exists for case (2), one approach is to first identify all candidate pairs that satisfy case (2). Under these conditions, if for every candidate pair the query result set consistently includes vertex ψ—meaning that a circle of radius at most r_x containing $core_{k_x}(q,\psi)$ can be identified within the region $C(q, 2\cdot r_x)$—then no valid refined parameter pair can be derived for the multiple vertices. For example, as shown in Fig. 1, when $q = L$, $\omega = Z$, and $\psi = E$, there is no parameter pair that would produce query results containing Z while excluding E. This implies that using parameter refinement methods, we can always inform users whether it's possible to include ω while excluding ψ, but we cannot always provide suitable parameter pairs.

IV. Algorithms for EOPE

Recall the framework depicted in Fig. 2, this section briefly describes ***our approaches*** proposed to solve the EOPE problem. Details can be found in our previous work [4].

A. The Effective Bounds of Refined k' and r'

As the core of the algorithms, the following property and lemma are presented. Property 1 reveals domination relationships between parameter pairs, indicating that certain dominated parameter pairs can be excluded from validation as they are inherently unqualified. Lemma 1 constrains the range of possible values that the parameters can take.

Property 1: For any parameter pairs (k', r'), (k', r''), and (k'', r''), the RB-k-core query results R_1 of $Q(k', r')$, R_2 of $Q(k', r'')$, and R_3 of $Q(k'', r'')$. If $r' < r''$ and $k' > k''$, there must exist $|R_1| \leq |R_2| \leq |R_3|$. Therefore, (k', r') dominates (k', r'') and (k', r'') dominates (k'', r'').

Lemma 1: Given a connected k-core G_k containing query vertex q and expected vertex ω in the graph G with the largest k, the bound of refined k' is $k'_{bound} = [k'_{\min}, k'_{\max}]$ where $k'_{\min} = 1$ and $k'_{\max} = \min\{k, cnes_{G_k}[\omega]\}$, the bound of r' is $r'_{bound} = [r'_{\min}, r'_{\max}]$ where $r'_{\min} = \max\{r, d(\omega,q)/2\}$ and $r'_{\max}$ is the radius of the MCC of $V(G_k)$.

B. Two Baseline Algorithms: PriorityR and HybridR

Based on the above, we propose two algorithms, **PriorityR** and **HybridR**, to further prune unqualified parameter pairs.

1) PriorityR: The first exploration strategy is to obtain k' within k'_{bound} for all $r' \in [r'_{\min}, r'_{\max}]$. Specifically, we fix the radius to r' and explore the optimal k' within k'_{bound}, where the possible values of r' are derived from the radius of all circles constructed using $V(G(\mathcal{C}(q, 2\cdot r_{\max})))$ based on the three-vertex approach. We then use an array $\mathcal{T}_{r'}$ (without duplicates) to store all r' in ascending order. Further pruning of unqualified r' in $\mathcal{T}_{r'}$ is performed based on a pruning strategy [4]. From all candidate pairs, the series of pairs with the smallest r' for the same k' is selected as the candidate parameter pairs $\mathcal{P}_a$. Finally, we validate all pairs in $\mathcal{P}_a$ using RotC+ and return the optimal parameter pair $\{r'_{opt}, k'_{opt}\}$.

2) HybridR: For the same explored k', only the parameter pair with the smallest r' could represent the optimal refined parameters. Using a dichotomy strategy, the problem of finding the smallest r' for each explored $k' \in k'_{bound}$ can be recursively divided into smaller subproblems based on different judgment conditions, as outlined below.

Subproblems Division: Our dichotomous strategy is realized by recursive methods. The recursive procedure is formalized as $Explore(low, high, \mathcal{T}_{r'}, k'_{bound}, \mathcal{P}_a)$, where low and $high$ are initialized as the head and tail of $\mathcal{T}_{r'}$. Each time *Explore* is called, the algorithm obtains r' in bisection on the ordered sequence $\mathcal{T}_{r'}$, i.e., $mid = (low + high)/2$, $r' = \mathcal{T}_{r'}[mid]$, and explores the refined k' for fixed r' within $[k'_{\min}, k'_{\max}]$. The recursion ends at $mid - low \leq 1$, and $\mathcal{P}_a$ is updated. Otherwise different subproblems are divided according to different judgment conditions. (1) If $k'_{bound}[0] == k'_{bound}[1]$ or $k' > k'_{bound}[1]$, r' contracts to the left, $high = mid$, $k'_{bound}[1] = k'$, and *Explore* is called; (2) If

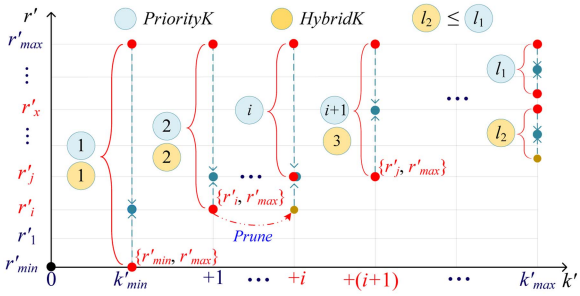


Fig. 3. Bounds for PriorityK and HybridK.

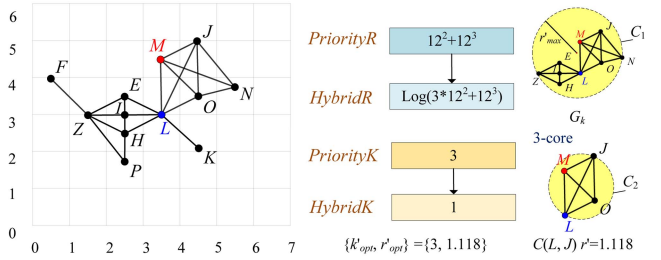


Fig. 4. Explain EOPE.

(1) is not satisfied and $k' \le k'_{bound}[0]$, r' contracts to the right, $low = mid$, $k'_{bound}[0] = k'$, and *Explore* is called; (3) If neither (1) nor (2) are satisfied, the recursive process divides into two subproblems, and r' contracts to both left and right, (1) and (2) are called once.

Based on the subproblems division, HybridR transforms the original sequential exploration in $\mathcal{T}_{r'}$ into a half-exploration approach, significantly improving exploration efficiency.

C. Two Effective Algorithms: PriorityK and HybridK

Motivated by the domination relationship established in Property 1 and the effective bounds of refined k', we propose our efficient exploration algorithms, **PriorityK** and **HybridK.**

1) PriorityK: It is evident that the possible values of k' are finite integers. This allows us to explore the minimum r' within the range $r'_{bound} = [r'_{\min}, r'_{\max}]$ for each $k' \in [k'_{\min}, k'_{\max}]$. As illustrated in Fig. 3, when k' is fixed at $k'_{\min}$, we explore in $r'_{bound} = [r'_{\min}, r'_{\max}]$ to determine the corresponding minimum radius r'_i. Subsequently, r'_{bound} is updated to $[r'_i, r'_{\max}]$. When $k' = k'_{\min} + 1$, the exploration of r' proceeds within the updated r'_{bound}. As k' increases incrementally, r'_{bound} is continuously narrowed until all candidate pairs are identified. In summary, PriorityK sequentially explores the minimum r' for each $k' \in [k'_{\min}, k'_{\max}]$, adding parameter pairs $\{k', r'\}$ to $\mathcal{P}_a$ while progressively tightening the range of possible r' for each k'. Finally, all pairs in $\mathcal{P}_a$ are validated using RotC+, and the optimal parameter pair $\{r'_{opt}, k'_{opt}\}$ is returned.

2) HybridK: Since multiple k' values may share a single r', as illustrated in Fig. 3, where $[k'_{\min}, k'_{\min} + i]$ share the same r'_i, PriorityK addresses this by first exploring the optimal r' for each $k' \in [k'_{\min}, k'_{\max}]$ and then preserving the optimal refined parameter pairs based on the domination relationship among the pairs. However, this approach incurs unnecessary time costs for exploring certain k' values. To address this limitation, we propose the following important theorem.

Theorem 1: Given a refined k'_i, the corresponding explored optimal refined r' is r'_i for k'_i, if the corresponding optimal refined k' can be explored as $k'_j (k'_j > k'_i)$ for r'_i, then for any k' in the range (k'_i, k'_j), the corresponding explored optimal refined r' must be r'_i, that is, the parameter pairs constructed by k' in the range $[k'_i, k'_j)$ and r'_i are dominated by $\{k'_j, r'_i\}$ based on Property 1, which do not need to be explored.

Based on Theorem 1, as illustrated in Fig. 3, the core idea of HybridK is to first obtain the optimal r'_i for $k'_{\min}$, while simultaneously exploring $k'_{\min} + i$ as the maximum k' for the expected results when r' is determined as r'_i. In other words, starting from $k'_{\min}$, we derive the candidate parameter pairs $\{k'_{\min} + i, r'_j\}$ in a single exploration, eliminating the need for $i + 1$ rounds of operations.

The summary of algorithms for EOPE: As illustrated in Fig. 4, we summarize the algorithms proposed for the EOPE problem. On the left side of the figure, given a graph G, $q = L$, $\omega = M$, and $Q(3, 1)$, the PriorityR algorithm identifies a G_k within C_1 and determines $r'_{\max}$. This implies that all vertices of the graph G are involved in the execution of PriorityR. Consequently, we need to explore the optimal k' for $12^2 + 12^3 = 1872$ parameter r' values and form parameter pairs, which is computationally intensive. To address this, we developed the HybridR algorithm, which reduces this number to 4 using a dichotomous strategy. Given that the number of possible k' values is typically a small integer, PriorityK in this example explores the optimal r' for 3 parameters k' ($k' \in [1, 3]$) and forms parameter pairs. However, this number can be further reduced by leveraging the domination relationships between parameter pairs, as demonstrated by property 1. HybridK is designed with this in mind. For instance, when exploring the optimal r' for $k' = 1$, we identify the circle C_2, which contains a $core_3(M, L)$. This allows us to directly obtain the parameter pair $\{3, 1.118\}$, bypassing the exploration of $k' \in [2, 3]$.

D. Hierarchical Coreness R-Tree

For our exploration algorithms, it consists of the following indispensable steps. (1) Constructing circles. (2) Selecting vertices in the circles to construct subgraphs. (3) Performing core decomposition and connectivity judgment. (4) Checking whether there exists a $core_{k'}(q, \omega)$. All previous work in this paper has focused on pruning the vertices used to construct the circles. However, after a series of prunings, there may still some unqualified circles are verified. In this section, therefore, in combination with R-Tree [11], we propose an index, called hierarchical coreness R-Tree (HCR-Tree), to prune some unqualified circles before constructing subgraphs.

The core idea of the HCR-Tree is that, for each node in the R-Tree, traversing from the root to the leaf, we extract all vertices within the node's region and construct a subgraph of the graph G. The coreness of these vertices is then computed and stored. Subsequently, for any given circle, the HCR-Tree can be utilized to determine the upper bound of the coreness of the vertices contained within that circle. This upper bound is used to assess

whether the circle can be pruned. The time complexity of the search operation using the HCR-Tree does not exceed that of the R-Tree. Details are provided in [4].

V. Exploring Optimal Refined Parameters k' and r' Simultaneously for EOPU

Recall the explanation framework: our goal is to obtain a parameter pair $Q'(k', r')$ such that, compared to other possible parameter pairs, the result of the RB-k-core query executed with Q' maximally preserves the original query results while ensuring that the query results exclude the unnecessary vertex ψ. To achieve this goal, we first establish bounds for the possible values of the refined parameter pairs and propose a basic solution. Next, we introduce our effective solution. Finally, we present an efficient method to quickly identify the optimal parameter pair from the explored candidates. Notably, our proposed HCR-Tree remains applicable to the EOPU problem and is utilized in a manner consistent with our previous presentation.

A. The Effective Bounds and Basic Solution

First we give upper and lower bounds for k' and r', determining the range of possible values for the refined parameters.

Lemma 2: Given an initial query $Q(k, r)$, a subgraph $G_s = G(\mathcal{C}(q, 2 \cdot r) \cap \mathcal{C}(\psi, 2 \cdot r))$, the bound of refined k' is $k'_{bound} = [k'_{\min}, k'_{\max}]$, where $k'_{\min} = k$ and $k'_{\max} = \min\{cnes_{G_s}[\psi], cnes_{G_s}[q]\} + 1$. Similarly, the bound of the refined r' is $r'_{bound} = [r'_{\min}, r'_{\max}]$, where $r'_{\min} = d(\psi, q)/2$ and $r'_{\max} = r$.

Proof: The values of $k'_{\min}$ and $r'_{\max}$ are set to the initial k and r, respectively, for obvious reasons. For $k'_{\max}$, G_s represents the entire search space when $r' = r'_{\max}$, and any k' greater than or equal to $k'_{\max}$ will cause the exclusion of either ψ or q. For $r'_{\min}$, any r' smaller than $r'_{\min}$ will result in q and ψ not being contained within a circle of radius r'.

After determining the bounds of the parameters, we consider an idea that combines the set of circles $\mathcal{C}_T s$ generated by the three-vertex method within the range $\mathcal{C}(q, 2 \cdot r'_{\max})$. For all $k' \in k'_{bound}$, we identify circles in $\mathcal{C}_T s$ that do not contain $core_{k'}(q, \psi)$, record the largest radius r', and form a parameter pair (k', r') with k'. Finally, we verify the result sizes of these parameter pairs and retain the largest one as the final result. However, a problem arises with the r' values obtained in this manner: there may exist $core_{k'}(q, \psi)$ in circles of smaller radius. This implies that, although our ultimate goal is to find query parameters that exclude unnecessary vertices from the query results, during the exploration phase, we need to identify circles from $\mathcal{C}_T s$ that contain $core_{k'}(q, \psi)$. The radius of these circles, combined with k', form a large set of candidate parameter pairs. Not all of these pairs are valuable, necessitating further pruning. An important property for this purpose is described below.

Property 2: For any parameter pairs (k', r'), (k', r''), and (k'', r'), where $r', r'' \in r'_{bound}$ and $k', k'' \in k'_{bound}$ (as established in Lemma 2), let R_1, R_2, and R_3 denote the RB-k-core query results of $Q'_1(k', r')$, $Q'_2(k', r'')$, and $Q'_3(k'', r')$, respectively. If $r' < r''$ and $k' > k''$, it necessarily follows that $R_1 \subseteq R_2 \subseteq R_3$.

Algorithm 1: BS(q, ψ, k, r).

Input: A graph $G(V, E)$, a HCR-Tree built with $G(V, E)$, a query vertex q, a query $Q(k, r)$, an unexpected vertex ψ
Output: Optimal refined parameter pair $\{k'_{opt}, r'_{opt}\}$

```
1  G_s ← a core_k(q, ψ) ∈ G(C(q, 2·r) ∩ C(ψ, 2·r));
2  initialize Pa, k'_bound, C_T s, r' = r;
3  foreach k' ∈ k'_bound do
4      foreach C(c, r'') ∈ C_T s do
5          if there exists core_{k'}(q, ψ) within C(c, r'') and
             r'' < r' then
6              Pa.k' = {k', r'' − σ};
7          else
8              continue;
9  {k'_opt, r'_opt} ← verify the parameter pairs in Pa;
10 return {k'_opt, r'_opt};
```

The meaning of Property 2 is that if certain parameter pairs satisfy the relationship mentioned, then the query results obtained from these pairs exhibit a containing relation. This containing relation serves two purposes. First, if some parameter pairs satisfy Property 2, we can always retain the smallest query results among them. In other words, for a series of remaining parameter pairs with the same k', if we identify the pair with the smallest r', then reducing r' by any value ensures that the unnecessary vertices be discarded. Second, leveraging this property, we propose the basic solution.

Algorithm 1 outlines the details of algorithm BS. The process begins by initializing the graph G_s, the array Pa, the bounds k'_{bound}, the set of three-vertex circles $\mathcal{C}_T s$, and the radius r'. Here, Pa is an array storing pairs of $\{k', r'\}$, and for convenience, we use $Pa.i$ to denote the pairs in the array where the k' is i (Lines 1-2). The set $\mathcal{C}_T s$ consists of circles constructed using the three-vertex method based on vertices in G_s. For each $k' \in k'_{bound}$, we identify all circles in $\mathcal{C}_T s$ that contain $core_{k'}(q, \psi)$ and record the smallest radius among them. Subtracting r' by σ ensures that ψ is discarded, where σ is a minimal floating-point value chosen to maintain computational accuracy. Given that our experiments are conducted on real-world datasets, we define $\sigma = 0.0001$ km, which introduces negligible additional vertex loss. We store all such pairs $\{k', r'' - \sigma\}$ in Pa and select the pair from Pa that retains the most original query results as the optimal refined parameter pair (Lines 3-10). Notably, for each k', we identify the circle of minimum radius from $\mathcal{C}_T s$ that contains $core_{k'}(q, \psi)$. Any reduction in the radius of this circle may result in the exclusion of ψ. However, a special case arises where discarding ψ causes the coreness of the query vertex q to violate the cohesiveness constraint, leading to the exclusion of q as well. In such cases, we directly remove the corresponding parameter pair.

Time Complexity: The size of $V(G_s)$ is defined as n, and there are $C \cdot (n^2 + n^3)$ circles to be constructed, where $C = (k'_{\max} - k'_{\min} + 1)$. The average time cost of core decomposition is defined as $O(n' + m')$, where $n' \ll |V(G_s)|$ and $m' \ll |E(G_s)|$. Thus, the time complexity of BS is $O(C \cdot (n^2 + n^3) \cdot (m' + n'))$.

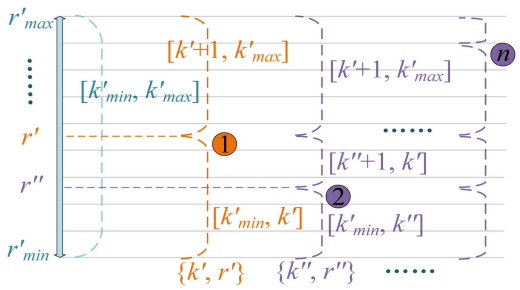


Fig. 5. Segmentation strategy.

B. The Segmentation Algorithm: SA

The basic solution requires constructing three-vertex circles for each k' in k'_{bound} and searching for subgraphs within each circle. This implies that $(k'_{\max} - k'_{\min} + 1) \cdot (|V(G_s)|^2 + |V(G_s)|^3)$ circles must be constructed and utilized. Although this is not an NP-hard problem, it is still challenging to solve in constant time [13]. To address this issue, we next present our segmentation algorithm.

The segmentation algorithm is based on the idea of constructing three-vertex circles $\mathcal{C}_T s$ using all vertices within $G_s = core_k(q, \psi) \in G(\mathcal{C}(q, 2 \cdot r) \cap \mathcal{C}(\psi, 2 \cdot r))$. It then computes the connected k'-core of $\mathcal{C}_T s$ that contains both q and ψ with the largest possible k'. The radius of these circles and their corresponding k' values form a series of parameter pairs, among which the final optimal parameter pair is identified. This implies that only one round of three-vertex circle construction is required, compared to BS. The sets of vertices used to construct the three-vertex circles are denoted as U_1, U_2, and U_3. Initially, $U_1 = U_2 = U_3 = V(G_s)$.

Building on this foundation, we design a segmentation strategy and a series of pruning strategies. The segmentation strategy is outlined as follows.

Segmentation Strategy: Given a set KR, which stores tuples $\{k_i, \{r_{\min_i}, r_{\max_i}\}\}$, where $k_i = k'_{\min} + i$, $r_{\min_i} = r'_{\min}$, and $r_{\max_i} = r'_{\max}$ $(i \in [0, n]$, $n = k'_{\max} - k'_{\min})$. For each tuple $\{k_i, \{r_{\min_i}, r_{\max_i}\}\}$, k_i represents values in the range $[k'_{\min}, k'_{\max}]$, and $\{r_{\min_i}, r_{\max_i}\}$ denotes the range of values that can form a refined parameter pair with k_i. Initially, for any $k_i \in KR$, there exists some $r_j \in [r_{\min_i}, r_{\max_i}]$ $(j \in [0, |\mathcal{C}_T s|])$ that can form parameter pairs with it. Based on Property 2, if $\exists \mathcal{C}(c, r') \in \mathcal{C}_T s$ such that there exists a $cnes_{k'}(q, \psi)$ in $G(\mathcal{C})$, where k' is the maximum possible value and $k' \in [k'_{\min}, k'_{\max}]$, then all $r_{\max_i}$ are updated to $\min\{r_{\max_i}, r'\}$ for $i \in [0, k' - k_0]$, and all $r_{\min_i}$ are updated to $\max\{r_{\min_i}, r'\}$ for $i \in [k' - k_0 + 1, n]$.

Fig. 5 illustrates an example of the segmentation strategy. Initially, for any $k_i \in [k'_{\min}, k'_{\max}]$, the radius of the circles that can form a refined parameter pair with it lie within the range $[r'_{\min}, r'_{\max}]$. First, we identify a circle $\mathcal{C}'(c', r') \in \mathcal{C}_T s$ and its corresponding $cnes_{k'}(q, \psi)$, forming a parameter pair $\{k', r'\}$ where $k' \in [k'_{\min}, k'_{\max}]$ and $r' \in [r'_{\min}, r'_{\max}]$, as shown in the figure at position "1". For all $k_i \in [k'_{\min}, k']$, their corresponding range $[r'_{\min}, r'_{\max}]$ in KR is updated to $\{r'_{\min}, r'\}$. For all $k_i \in [k' + 1, k'_{\max}]$, their corresponding range $[r'_{\min}, r'_{\max}]$ in KR is updated to $\{r', r'_{\max}\}$. This process continues, progressively updating the values in KR.

The proposed segmentation strategy implies that for each $k' \in k'_{bound}$, the range of possible values of r' that can form a parameter pair with k' is continuously shrinking. By leveraging this property, we prune the sets U_1, U_2, and U_3 involved in the construction of the three-vertex circles. Here, the vertices in U_1, U_2, and U_3 are sorted in ascending order based on their distance from q.

Pruning strategy of U_1 (called $n\varphi_1$): Given a vertex $u \in U_1$, let $r_t = \max\{r_{\max_i}\}$ where $i \in [0, n]$. The vertex u must satisfy the condition $\max\{d(q, u), d(\psi, u)\}/2 \leq r_t$.

An important lemma [13] is shown next to assist in pruning.

Lemma 3: Given a circle $\mathcal{C}(t, r_t)$ and two boundary vertices v_i and v_j of the circle, the largest distance, dis, between v_i and v_j must satisfy $dis \in [\sqrt{3} \cdot r_t, 2 \cdot r_t]$.

Pruning strategy of U_2 (called $n\varphi_2$): Given a vertex $u \in U_1$, $v \in U_2$; $r_t = \max\{r_{\max_i}\}$, $r_u = \min\{r_{\min_i}\}$ $(i \in [0, n])$. (1) The vertex v needs to satisfy $\sqrt{3} \cdot \max\{r_u, max\{d(q, u), d(\psi, u)\}/2\} < d(u, v) \leq 2 \cdot r_t$; (2) Given the current KR, $k_c = \min\{cnes_{G_s}[u], cnes_{G_s}[v]\}$, when a three-vertex circle $\mathcal{C}_T(u, v)$ is constructed, $r_u = \min\{r_{\min_i}\}(i \in [0, k_c\text{-}k_0])$, the vertex v needs to satisfy $d(u, v)/2 \geq r_u$.

We constrain that the distance between the vertices in U_1 and U_2 is the largest among all the vertices involved in constructing the three-vertex circle. Therefore, $n\varphi_2$ can be derived based on Lemma 3 (1). $n\varphi_2$ (2) indicates that for any k', if we want to search for k'-cores in G_s, vertices with a coreness less than k' can be removed directly. Additionally, $k_c = \min\{cnes_{G_s}[u], cnes_{G_s}[v]\}$ means that vertices u and v are involved in the search of the k_c-core at most, so we can prune v based on the lower bound of the corresponding radius.

Pruning strategy of U_3 (called $n\varphi_3$): Given a vertex $u \in U_1$, $v \in U_2$, and $x \in U_3$, when constructing a circle $\mathcal{C}_T(u, v, x)$, the following conditions must be satisfied: (1) The vertex x must satisfy $\max\{d(x, u), d(x, v)\} \leq d(u, v)$; (2) Given the current KR, let $k_c = \min\{cnes_{G_s}[u], cnes_{G_s}[v], cnes_{G_s}[x]\}$. When a circle $\mathcal{C}_T(u, v, x)$ is constructed, let $r_u = \min\{r_{\min_i}\}$ $(i \in [0, k_c - k_0])$. The radius r_c of $\mathcal{C}_T(u, v, x)$ must satisfy $r_c > r_u$.

$n\varphi_3$ (1) can be applied for reasons that are easy to understand. The derivation of $n\varphi_3$ (2) is similar to that of $n\varphi_2$ (2).

Then, based on the lemma below from our previous work, the termination strategy is described.

Lemma 4: Given a query vertex q, a vertex $u \in U_1$, and two vertices $v_i, v_j \in U_2$, if $d(q, v_i) \leq d(q, v_j)$, the refined r' returned by $\mathcal{C}_T(u, v_i)$ must be smaller than the refined r'' returned by $\mathcal{C}_T(u, v_j)$. In other words, if a valid refined r' is obtained from $\mathcal{C}_T(u, v_i)$, the circles constructed by u and the remaining vertices (such as v_j) in U_2 can be safely pruned.

Termination condition ($\mathcal{TC}$). (1) First, we can terminate the U_2 loop early according to Lemma 4. (2) In the U_1 loop, if $\mathcal{RP}_1$ is satisfied, for $u \in U_1$, when $d(u, q)/2 \geq r_t$, where $r_t = \max\{r_{\max_i}\}$ $(i \in [0, n])$, the loop can be terminated early. The reason is that $d(u, q)/2$ represents the smallest radius that ensures both q and ψ are within a $\mathcal{C}_T$ constructed using u.

Algorithm 2: SA(q, ψ, k, r).

Input: A graph $G(V, E)$, a HCR-Tree built with $G(V, E)$, a query vertex q, a query $Q(k, r)$, an unexpected vertex ψ

Output: Optimal refined parameter pair $\{k'_{opt}, r'_{opt}\}$

1 $G_s \leftarrow$ a $core_k(q, \psi) \in G(\mathcal{C}(q, 2 \cdot r) \cap \mathcal{C}(\psi, 2 \cdot r))$;
2 initialize KR, U_1, U_2, U_3;
3 **foreach** $u \in U_1$ **do**
4 prune according to $n\varphi_1$ and $\mathcal{TC}$ (2);
5 **foreach** $v \in U_2$ **do**
6 prune according to $n\varphi_2$;
7 **if** *better result exist in* $\mathcal{C}_T(u, v)$ **then**
8 update KR; *segmentation strategy*\
9 break; \ * $\mathcal{TC}$ (1) *\
10 **else**
11 **foreach** $x \in U_3$ **do**
12 prune according to $n\varphi_3(1)$;
13 **if** *the triangle constructed by* $\{u, v, x\}$ *is not an obtuse triangle* **then**
14 constructe $\mathcal{C}(t, r) = \mathcal{C}_T(u, v, x)$;
15 **if** $d(t, q(or\ \psi)) > r$ *or* $n\varphi_3(2)$ **then**
16 continue;
17 **else if** *find better results* **then**
18 update KR;
19 **else**
20 continue;
21 $\{k'_{opt}, r'_{opt}\} \leftarrow$ verify the parameter pairs in KR;
22 **return** $\{k'_{opt}, r'_{opt}\}$;

Combining the above strategies, algorithm SA is presented in Algorithm 2. First, we initialize G_s and KR according to the segmentation strategy (Lines 1-2). Next, U_1 is pruned based on $n\varphi_1$, and the loop termination is determined using $\mathcal{TC}(2)$ (Lines 3-4). Then, for the U_2 loop, U_2 is pruned based on $n\varphi_2$. If there exists $cnes_{k'}(q, \psi)$ in $\mathcal{C}_T(u, v)$, where k' is the maximum possible value, KR is updated according to the segmentation strategy, and termination is determined using $\mathcal{TC}(1)$ (Lines 5-9). Otherwise, we proceed to the next loop and prune based on $n\varphi_3(1)$. If the triangle formed by the vertices $\{u, v, x\}$ is not an obtuse triangle, we construct the circle $\mathcal{C}_T(u, v, x)$, check whether it contains both q and ψ, and determine whether to prune the circle based on $n\varphi_3(2)$. If pruning is not triggered and a better result exists in $\mathcal{C}_T(u, v, x)$, KR is updated again based on segmentation strategy (Lines 10-20). Finally, all $\{k_i, r_{\max_i} - \sigma\}$ $(i \in [0, n])$ are validated to identify the pair that retains the most original query results, denoted as $\{k'_{opt}, r'_{opt}\}$ (Lines 21-22).

Example 1: Fig. 6 illustrates an example of Algorithm SA. Given a query vertex L, an unnecessary vertex $\psi = Z$, and a query $Q(3, 1)$, we first initialize G_s as a subgraph consisting of vertices $\{L, H, I, E, P, Z\}$, KR as $\{3, \{1, 1\}\}$, and U_1, U_2, and U_3 as shown in the top right of the figure. When $u = L$, all vertices $\{I, E, H, P\}$ in U_2 are pruned due to $n\varphi_2(1)$. Additionally, because a $core_3(L, Z)$ exists within $\mathcal{C}(L, Z)$, all vertices in U_3

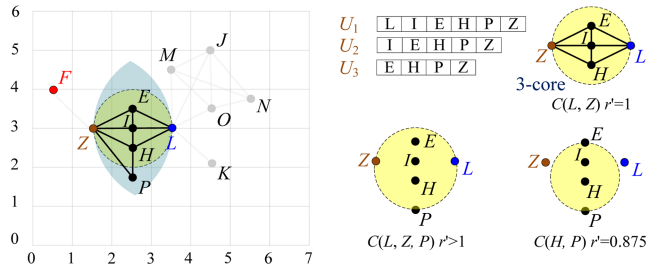


Fig. 6. An example of SA.

are pruned according to $(\mathcal{TC})(1)$, preventing the construction of circles like $\mathcal{C}(L, Z, P)$. When $u = I$, all vertices in U_2 are pruned due to $n\varphi_2(1)$. When $u = H$, $\mathcal{C}(H, P)$ is pruned because it does not contain L and Z. Iterating through this process, the optimal refined parameter pair $\{k'_{opt}, r'_{opt}\} = \{3, 0.999\}$ is finally obtained.

Time Complexity Analysis: The time complexity of Algorithm SA is derived as follows. Let $n = |V(G_s)|$ and $m = |E(G_s)|$ denote the numbers of vertices and edges in the pruned subgraph G_s, respectively. After applying the pruning strategies $n\varphi_1, n\varphi_2$, and $n\varphi_3$, the vertex sets U_1, U_2, and U_3 are reduced to sizes N_1, N_2, and N_3, respectively. Let N_1/n, N_2/n, and N_3/n denote the pruning ratios. Empirically, N_1/n, N_2/n, and N_3/n are less than 1 (see Table VIII for specific values across different datasets). Within the three-level nested loop (lines 3-20), each iteration processes a candidate triple (u, v, x) where $u \in U_1$, $v \in U_2$, and $x \in U_3$. The dominant cost within the loop is constructing the subgraph $G(C(t, r))$ and performing core decomposition (lines 13-18), which costs $O(n' + m')$, where n' and m' are the numbers of vertices and edges in the constructed subgraph, respectively. Typically, $n' \ll n$ and $m' \ll m$[3]. Therefore, the time complexity of SA is $O(N_1 \cdot N_2 \cdot N_3 \cdot (n' + m'))$, where $\alpha = N_1 \cdot N_2 \cdot N_3/n^3$ is the overall pruning ratio coefficient. Experimental results (see Table VIII) demonstrate that α is in the range of 10^{-6} to 10^{-3} across various datasets, which explains the practical efficiency of SA despite its cubic worst-case time complexity.

C. Fast Verification of Optimal Parameter Pairs

All algorithms proposed for the EOPE and EOPU problems share a key step: *verifying the parameter pairs in* KR_e (KR_u), where KR_e (KR_u) is defined as the set of remaining candidate parameter pairs for the EOPE (EOPU) problem. Previously, we verified the optimal parameter pair by calling the RotC+ algorithm once for each parameter pair in the set, counting the size of the result, and finally returning the best parameter pair (the smallest size for the EOPE problem and the largest size for the EOPU problem). As described in Section I, the size of the result is the primary concern, making the use of RotC+ for obtaining the optimal parameter pair inefficient. To further improve the efficiency of acquiring the optimal parameter pair, this section introduces a fast parameter pair verification algorithm.

Although we have previously briefly described the idea of the RotC+ algorithm, to better explain our proposed parameter pair validation method, we now detail the flow of the RotC+ algorithm. Below is one of the pruning strategies used by the RotC+ algorithm.

Pole-Pruning [3]: Given a vertex v and a positive integer k, before constructing a binary-vertex-bounded circle using v, verify whether there exists a $core_k(q, \omega)$ within $\mathcal{C}(v, 2 \cdot r)$. If not, the vertex v can be pruned.

RotC+ [3]: Given a graph $G(V, E)$, a query vertex q, a query radius r, and a positive integer k, the RotC+ algorithm proceeds as follows: (1) Extract a subgraph of G within the range $\mathcal{C}(q, 2 \cdot r)$, denoted as G_k, where G_k is a $core_k(q)$; (2) Select any vertex in G_k as a pole p. If the pole is not pruned by *Pole-Pruning*, construct binary-vertex-bounded circles using p and all other vertices of G_k; (3) Construct a polar coordinate system with p as the origin and sort the constructed circles in ascending order of their polar angles; (4) Perform two rounds of pruning on the ascending-ordered circles, then determine whether an RB-k-core exists in the remaining circles; (5) Iterate the above process until all possibilities are exhausted.

The primary purpose of RotC+ is to search for all possible RB-k-core results, whereas our work focuses more on the number of vertices in the query result. Given the potentially high overlap between RB-k-core search results, it is inefficient to first search all RB-k-cores and then calculate the number of vertices in the query results. This means that directly using RotC+ to determine the best refined query parameters is not efficient, especially when the overlap of query results is extremely high. Therefore, we propose the following method to quickly obtain the optimal parameter pairs.

Fast parameter pairs verification (FA): (1) We directly compute the number of vertices $counts$ in the results of each refined RB-k-core query instead of recording all RB-k-core subgraphs. We also compute the number of vertices P_r pruned by *Pole-Pruning*. (2) Since we need to evaluate multiple sets of recommendation queries in KR_e (KR_u), we first record the number of vertices in $core_{G_{k'}}(q)$ for each $\{k', r'\} \in KR_e$ (KR_u), where $G_{k'}$ is a subgraph of G within $\mathcal{C}(q, 2 \cdot r')$. We then sort KR_e in ascending order and KR_u in descending order based on these numbers, and process the pairs in KR_e (KR_u) sequentially. (3) For KR_e, we define the size of the query result corresponding to the current optimal parameter pairs as $S_{\max}$. At any point, if $counts > S_{\max}$, the verification of the current pair terminates. For KR_u, we define the size of the query result corresponding to the current optimal parameter pairs as $S_{\min}$. At any point, if $|V(G_{k'})| - P_r < S_{\min}$, the verification of the current pair terminates. (4) For any $\mathcal{C}_B$, if the vertices used to construct it are either pruned by *Pole-Pruning* or recorded as query results, then $\mathcal{C}_B$ is pruned. At any point, if $P_r + counts = |V(G_{k'})|$, the verification of the current pair can be terminated.

VI. Experiments

In this section, we evaluate the efficiency and effectiveness of the proposed exploration algorithms.

TABLE II
Datasets Used

Datasets	$\|V\|$	$\|E\|$	$\|d_{avg}\|$
Weeplace(WP)	15799	57065	7.22
Brightkite(BK)	51406	197167	7.67
Gowalla(GW)	107092	456830	8.53
Flickr(FK)	214698	2096306	19.5
Foursquare(FS)	2127093	8640352	8.12

TABLE III
Parameter Settings

Parameters	Range	Default
k	4, 7, 10, 13, 16	10
$r(km)$	1, 3, 5, 7, 9	5
n	20%,40%,60%,80%,100%	100%

A. Experiment Setting

Algorithms: All algorithms are implemented in C++ and executed on a PC running Ubuntu 20.04, equipped with a 3.9 GHz CPU and 256 GB RAM. (1) Initially, HybridR and HybridK are implemented using R-Tree and denoted as HybridR-R and HybridK-R, respectively. We then enhance all exploration algorithms by utilizing HCR-Tree, resulting in PriorityR, PriorityK, HybridR, and HybridK. To determine the optimal refined query from candidate parameter pairs, the RotC+ algorithm is used to verify these pairs. Additionally, the single-parameter refining algorithms REFK and REFR [8] are implemented to compare the quality of the explored refined parameters. Subsequently, KSize, RSize, and MixSize represent the size of the refined query results obtained by REFK, REFR, and HybridR/HybridK, respectively. (2) To address the EOPU problem, 1) we implement the algorithms BS and SA. 2) Furthermore, the most effective solution for the EOPE problem, HybridK, is extended to solve the EOPU problem and is denoted as HU. This is achieved by initializing HybridK with r'_{bound} and k'_{bound} using the parameter bounds provided by Lemma 2, then exploring r' for each $k' \in k'_{bound}$. 3) We also implement algorithms BS, SA, and HU using R-Tree, denoted as BS-R, SA-R, and HU-R, respectively, to evaluate the efficiency of HCR-Tree in solving the EOPU problem. 4) Finally, the FA algorithm is implemented and compared with RotC+. If an algorithm cannot complete within 10^7 ms, it is denoted as ENF.

Datasets: Our experiments use five real-world datasets Weeplace,[1] Brightkite[2], Gowalla[2] [14], Flickr,[3] and Foursquare[2] [15]. The details of the datasets are given in Table II. where $|V|$ is the number of vertices, $|E|$ is the number of edges, and $|d_{avg}|$ represents the average degree.

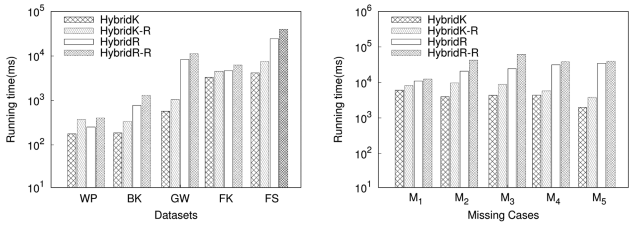
Table III shows the range and default values of the parameters, k is the cohesiveness constraint, r is the query radius, n is the dataset size.

To evaluate the performance of the algorithms, we: (1) Set the parameters to default values and generate 100 queries for

[1]https://www.yongliu.org/datasets/
[2]https://www.comp.hkbu.edu.hk/
[3]https://www.flickr.com/

TABLE IV
AVERAGE RUNNING TIME OF ALGORITHMS

Time (ms) / Datasets / Algorithms	WP	BK	GW	FK	FS
HEFK	2029.43	19005.4	24586.2	37424.9	353150
HEFR	2249.4	28816.7	29168.1	173427.3	ENF
PriorityR	3654.34	52711.1	653630.8	60056.8	ENF
HybridR	252.28	770.86	8343.72	4672.18	24469.35
PriorityK	192.86	204.36	655.59	3461.88	4759.14
HybridK	**177.26**	**186.15**	**571.34**	**3289.86**	**4154.61**



(a) Different datasets (b) Missing cases in Foursquare

Fig. 7. Efficiency of HCR-Tree.

TABLE V
SIZE OF REFINED RESULTS UNDER DIFFERENT MISSING CASES

Size (#) / Missing cases / Datasets		M_1	M_2	M_3	M_4	M_5
Brightkite	Ksize	342	407	1182	⊗	⊗
	Rsize	509	415	⊗	1721	⊗
	MixSize	**247**	**306**	**1036**	**880**	**1861**
Gowalla	Ksize	1544	1994	4673	⊗	⊗
	Rsize	3239	12777	⊗	11931	⊗
	MixSize	**1184**	**1973**	**4126**	**2870**	**7600**
Flickr	Ksize	230	297	308	⊗	⊗
	Rsize	2284	5458	⊗	8655	⊗
	MixSize	**186**	**229**	**301**	**356**	**465**
Foursquare	Ksize	41731	39822	34114	⊗	⊗
	Rsize	59921	125101	⊗	98209	⊗
	MixSize	**33142**	**35513**	**29351**	**19153**	**35754**

each dataset by randomly selecting 100 different query vertices. Then, we randomly generate 10 different expected (unexpected) vertices for each query. For EOPE, the expected vertices are evenly distributed across missing cases 1)-5). (2) Set other parameters to default and vary the parameter k/r as shown in Table III to generate 10 query vertices for each k/r on each dataset separately, along with 10 expected (unexpected) vertices generated in the same manner as above. These experiments evaluate the effect of the parameter k/r.

B. Efficiency Assessment of EOPE Problem

This section evaluates the solutions proposed for solving the EOPE problem, and the algorithms mentioned in this section are all proposed for solving the EOPE problem, unless otherwise stated.

Exp-1: As seen in Table IV. We initially assess the efficacy of the proposed algorithms on different datasets. First, except for PriorityR, our mixed refining algorithms outperform the old single parameter refining algorithms HEFK and HEFR in terms of efficiency. Then, HybridR, compared to PriorityR, achieves efficiency advancements of between one and three orders of magnitude as anticipated. And PriorityK is quicker than HybridR. This trend exists because the value of k' is generally a small constant. HybridK proves to be the most effective, as its efficiency directly correlates with the number of instances of k' shared with a single r'. Overall, HybridK enhances efficiency by 1.05–1.2 times on average, as compared to PriorityK.

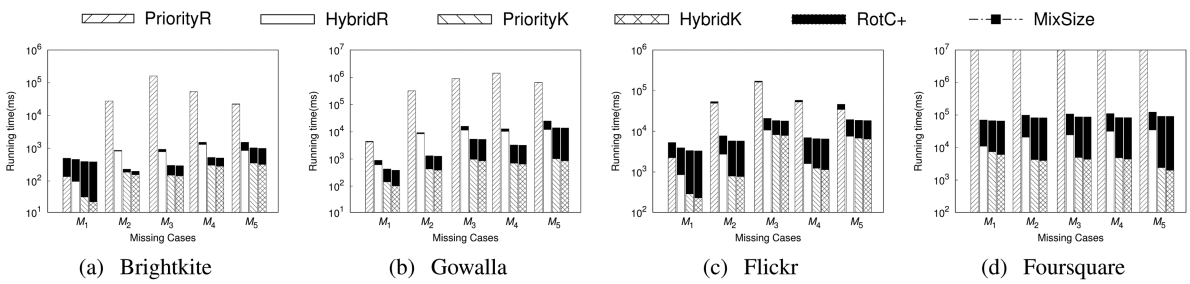
Exp-2: We assessed the efficiency of HCR-Tree, as evidenced in Fig. 7(a). Results indicated that HCR-Tree boosted HybridK efficiency by approximately 2-fold compared to HybridK-R on all datasets. The reason behind this is that HCR-tree prunes many circles. On all datasets, the efficiency of HybridR is improved by about 1.5 times compared to that of HybridR-R. In Fig. 7(b), we display the performance of HCR-Tree under different cases on the Foursquare dataset. HCR-Tree shows the greatest enhancements for HybridR in case 3 with a 2.5x increase in efficiency, 2x in case 2, and 1.13x in case 1. For case 2, HCR-Tree is the most efficient option for HybridK, resulting in a 2.4 times efficiency gain for the same rationale.

Exp-3: We evaluate the running time of the algorithms under different missing cases. As detailed in Fig. 8. The efficiency of HybridK and PriorityK is mainly affected by $k'_{\max}$-$k'_{\min}$, and the larger this value is, the more expensive HybridK and PriorityK is. The execution time of HybridK and PriorityK basically increases from M_1 to M_5. The $k'_{\max}$-$k'_{\min}$ is smallest in case M_1 and second smallest in M_2, so the efficiency of HybridK and PriorityK is highest in M_1 and second highest in M_2 as seen in Fig. 8(a)–(c). For cases M_3 to M_5, the difference of $k'_{\max}$-$k'_{\min}$ is not significant, so the efficiency of HybridK and PriorityK grows smoothly in Fig. 8(d). As shown in Fig. 8(a)–(d), the efficiency of PriorityR and HybridR mainly depends on the $|\mathcal{T}_{r'}|$, and the higher the $|\mathcal{T}_{r'}|$, the lower the efficiency. From M_1 to M_3, the $|\mathcal{T}_{r'}|$ increases, so the performance of PriorityR and HybridR shows an increasing trend. From M_3 to M_5, the $|\mathcal{T}_{r'}|$ and running time of PriorityR and HybridR decrease. The positive correlation between the enhancement of PriorityR's efficiency by HybridR and the difference between $|\mathcal{T}_{r'}|$ and $log_2|\mathcal{T}_{r'}|$, i.e, the larger $|\mathcal{T}_{r'}| - log_2|\mathcal{T}_{r'}|$, the more pronounced the difference in efficiency between HybridR and PriorityR, so from M_2 to M_4, HybridR improves PriorityR's efficiency the most. Finally, in any case, PriorityK is always more efficient than PriorityR.

Exp-4: We assessed the size of results of explored refined queries for the proposed algorithms across various missing cases. ⊗ means that the case cannot be explained by a certain algorithm. Table V shows that refining the parameters simultaneously has the best quality, i.e., MixSize is the smallest. Generally, RSize is the largest for each missing case, although there are some cases where RSize is smaller than KSize, and KSize is usually closer to MixSize. Increasing the radius generally results in more vertices appearing in the query results for an RB-k-core query, compared to decreasing k.

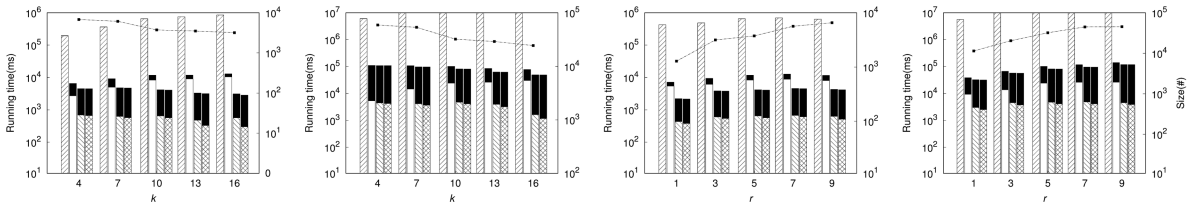
Exp-5: The impact of the parameters k and r on the performance of algorithms is shown in Fig. 9.

(1) Fig. 9(a) and (b) show the effect of parameter k. For PriorityR and HybridR, increasing k can result in higher values of $k'_{\max}$, and larger $k'_{\max}$ leads to higher $r'_{\max}$, considering that $\mathcal{T}'_r$ is generated from $\mathcal{C}(q, 2 \cdot r'_{\max})$, and larger $r'_{\max}$ leads to



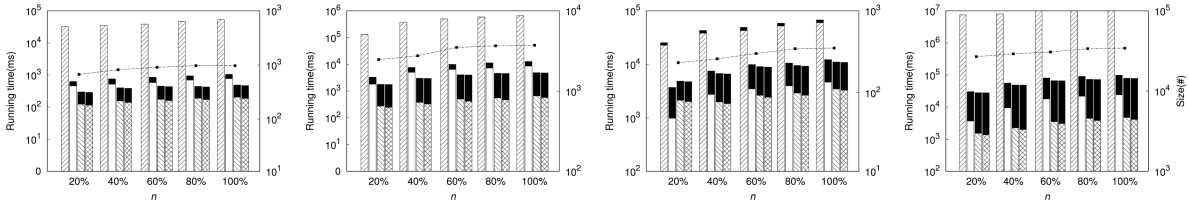
(a) Brightkite (b) Gowalla (c) Flickr (d) Foursquare

Fig. 8. Performance vs. missing cases.



(a) Gowalla (b) Foursquare (c) Gowalla (d) Foursquare

Fig. 9. Performance vs. parameters k and r.



(a) Brightkite (b) Gowalla (c) Flickr (d) Foursquare

Fig. 10. Performance vs. sizes of datasets.

larger $|\mathcal{T}_r'|$. The increase of k leads to an expansion in $|\mathcal{T}_r'|$, resulting in an increase in the running time of both PriorityR and HybridR from 4 to 16. Since larger values of k lead to fewer vertices that satisfy the cohesiveness, the efficiency of PriorityK and HybridK is improved as k gets larger. As k increases from 4 to 10, the number of vertices satisfying the cohesiveness decreases significantly, thus the largest changes in the result sizes occur during [4,10]. As k increases from 10 to 16, the trend slows down. (2) We assess how the parameter r impacts the algorithms. The accompanying Fig. 9(c) and (d) reveal a decline in efficiency for all algorithms as r increases. Moreover, as r grows, the result sizes also increase. This is due to the enlargement of vertices with an increase in r, which consequently demands the algorithms handle a greater volume of vertices.

Exp-6: Fig. 10 illustrates the impact of dataset size (ranging from 20% to 100%) on algorithms' running time and result size. The running time of each algorithm and the size of the results increase as the size of the dataset increases. Between 20% and 60%, the local density of the graph is significantly impacted by the addition of vertices and edges. Consequently, this leads to a more pronounced trend in results and running time growth. However, the growth rate of result size and running time slows down between 60% and 100%. In Fig. 10(c), when the size is 20% of the Flickr dataset, HybridR is the fastest due to the high degree of vertex overlap within Flickr, which, coupled with its relatively small number of vertices, leads to a reduced number of radius requiring verification.

C. Efficiency Assessment of EOPU Problem

In this section, we evaluate the algorithms developed for solving the EOPU problem. As all proposed algorithms are exact—ensuring they always find optimal parameter pairs when solutions exist—the experimental assessment primarily examines their runtime performance and scalability.

Exp-7: First, we compare the efficiency of algorithms for solving the EOPU problem. Table VI shows the average execution time of the algorithms BS, HU, and SA when the parameters are set to default. It can be observed that SA significantly outperforms BS in efficiency and is, on average, about 5 times more efficient than HU. This is because HU always attempts to explore r' for multiple k', resulting in a computational

TABLE VI
AVERAGE RUNNING TIME (EOPU)

Time (ms) / Datasets / Algorithms	WP	BK	GW	FK	FS
BS-R	4349.94	13152.81	158052.13	119752.13	ENF
HU-R	41.12	1244.47	7721.95	4156.88	33031.11
SA-R	4.94	145.11	685.32	786.09	5897.1
BS	1820.06	5159.99	58226.3	55828.5	ENF
HU	21.60	682.65	4494.73	3320.19	22378.8
SA	**4.54**	**107.41**	**547.38**	**616.54**	**4374.7**

TABLE VII
COMPARE ROTC+ AND FA

Time (ms) / Datasets / Algorithms	WP	BK	GW	FK	FS
RotC+(E)	846.4	1519.28	3519.69	7084.36	97877.2
FA(E)	**139.62**	**383.42**	**1430.6**	**2909.38**	**28271.8**
RotC+(U)	584.42	975.25	3744.87	5736.85	32224.9
FA(U)	**158.64**	**254.86**	**616.58**	**1929.23**	**14876.9**

TABLE VIII
PERCENTAGE OF N_i

Datasets	$\lvert N_1\rvert/\lvert V(G_s)\rvert$	$\lvert N_2\rvert/\lvert V(G_s)\rvert$	$\lvert N_3\rvert/\lvert V(G_s)\rvert$
Weeplace	0.81	0.29	0.03
Brightkite	0.42	0.25	0.01
Gowalla	0.47	0.18	0.01
Flickr	0.04	0.01	0.003
Foursquare	0.05	0.03	0.001

cost equivalent to multiple executions of SA. When solving the EOPU problem, we compare HybridK, the best algorithm for solving the EOPE problem, as shown in Table IV. SA demonstrates similar efficiency to HybridK on the FS dataset and outperforms HybridK on all other datasets. This indicates that, despite the increased complexity of the EOPU problem compared to the EOPE problem, our proposed SA algorithm for the EOPU problem exhibits advantages even over the best EOPE algorithm, HybridK. It is noteworthy that although BS demonstrates the lowest average efficiency, there exist specific individual queries for which BS may outperform HU in terms of computational efficiency. Overall, the experiment confirms that the SA algorithm is acceptably efficient for solving the EOPU problem.

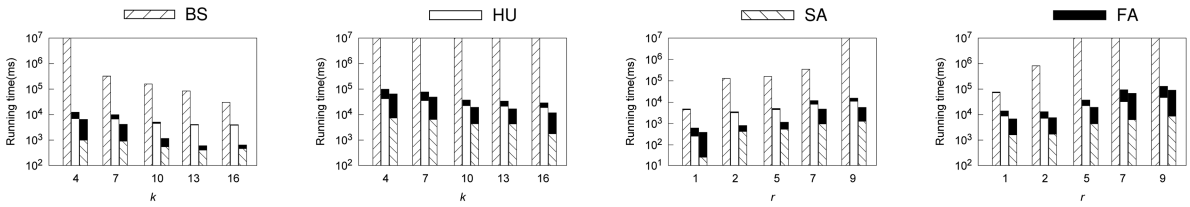
Exp-8: We evaluate the effectiveness of using HCR-Tree to solve the EOPU problem. As shown in Table VI, on five datasets, the efficiency improvement of BS over BS-R ranges from 2.14 to 2.7 times. The improvement of HU over HU-R ranges from 1.25 to 1.9 times, and the improvement of SA over SA-R ranges from 1.1 to 1.35 times. It is evident that while the efficiency of the algorithms improves from BS to SA, the effectiveness of HCR-Tree gradually decreases. This is because BS directly searches a large number of circles using HCR-Tree without pruning, whereas SA prunes many unqualified circles in advance. The more circles HCR-Tree searches, the more likely pruning is triggered, reducing overhead and increasing effectiveness. Overall, HCR-Tree achieves an average 1.26 times efficiency improvement over R-Tree for the best algorithm, SA, demonstrating its effectiveness.

Exp-9: Under the default parameter settings in Table III, Table VIII presents the average proportions of $|N_1|$, $|N_2|$, and $|N_3|$ relative to $|V(G_s)|$ across all experimental queries. First, we compared three pruning strategies and observed a gradual increase in the number of pruned vertices. This is because the spatial scope of the pruning functions progressively shrinks from $n\varphi_1$ to $n\varphi_3$. Across different datasets, pruning performance is the weakest on Weeplace due to its small dataset size and sparse spatial distribution, resulting in a limited initial vertex set $|V(G_s)|$. The best performance was achieved on Flickr and Foursquare, where the dense spatial distribution of data concentrates many vertices within small distances, enabling distance-based pruning strategies to excel. On Brightkite and Gowalla datasets, similar data distributions led to comparable pruning effectiveness. In summary, pruning effectiveness correlates with spatial vertex density: denser distributions and higher vertex counts lead to better pruning results. Furthermore, $n\varphi_1$, $n\varphi_2$, and $n\varphi_3$ demonstrate progressively stronger vertex pruning capabilities. In conclusion, the proposed pruning strategies demonstrate effectiveness.

Exp-10: Table VII compares the ability of RotC+ and FA to identify optimal refined parameter pairs from the explored parameter sets. RotC+(E) and FA(E) represent the efficiency comparison for solving the EOPE problem on each dataset, while RotC+(U) and FA(U) correspond to the efficiency for solving the EOPU problem. Due to our effective optimization strategies, FA consistently outperforms RotC+ in both EOPE and EOPU problems. Specifically, FA(E) achieves an average efficiency improvement of 3.68× over RotC+(E), and FA(U) achieves an average improvement of 3.74× over RotC+(U), demonstrating the effectiveness of the FA algorithm. Comparing FA(E) and FA(U), FA(U) generally incurs lower computational costs for two reasons: (1) the EOPU problem typically has fewer candidate parameter pairs than EOPE, and (2) the parameter pairs returned by EOPU impose stronger spatial and cohesiveness constraints, resulting in fewer vertices satisfying these constraints and participating in FA computations compared to EOPE. Overall, these results confirm the high effectiveness of the FA algorithm.

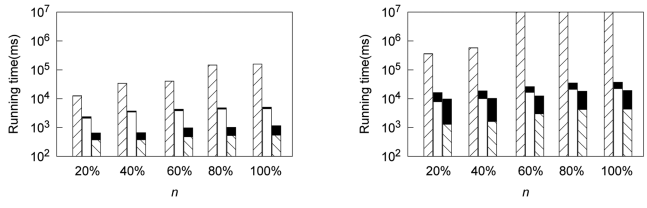
Exp-11: Fig. 11 demonstrates the effect of varying parameters k and r on algorithm efficiency. As shown in Fig. 11(a) and (b), the execution efficiency of BS, HU, and SA increases as k grows from 4 to 16. This is because the number of vertices satisfying the cohesiveness requirement decreases as k increases, reducing the number of vertices involved in algorithm execution. On the other hand, Fig. 11(c) and (d) show the effect of parameter r on algorithm efficiency. As r increases, the algorithms involve larger regions and more vertices, leading to a gradual decrease in efficiency. As shown in Fig. 11(c), the efficiency of the algorithms decreases significantly when r grows from 1 to 2 on the Gowalla dataset but remains relatively stable as r increases from 2 to 5. This is because the number of vertices meeting the default cohesiveness requirement is very small when r is 1. There is a sudden and large increase in this number when r grows to 2, while the change is more stable as r increases from 2 to 5.

Exp-12: Fig. 12 evaluates the effect of dataset size on the algorithms solving the EOPU problem. Overall, algorithm efficiency decreases as dataset size increases, consistent with our expectation that larger datasets require more time to search for



(a) Gowalla (b) Foursquare (c) Gowalla (d) Foursquare

Fig. 11. Performance vs. parameters k and r (EOPU).



(a) Gowalla (b) Foursquare

Fig. 12. Performance vs. size of datasets (EOPU).

results. As shown in Fig. 12(b), although BS fails to obtain results within the timeout period on the Foursquare dataset when the dataset size exceeds 60%, SA maintains its superiority, solving the problem with the highest efficiency even under these conditions.

VII. Related Work

Community search: Sozio et al. [16] first proposed community search problem to obtain a subgraph containing a given query vertex. Based on this, some studies have investigated the problem [13], [17], [18], [19]. Zhu et al. [20] proposed two spatial-aware indexes to search communities by spatial distance and social relations on a given rectangle. [21] proved that the problem of reliable community search on uncertain graphs is NP-hard and proposed some efficient algorithms. [22] presented a deferentially private algorithm to output private k-core decomposition statistics. The τ-cover problem of finding maximal clique was proposed by [23]. [24] considered the problem of continuous geo-social group monitoring over moving users on LBSN. [25] investigated keywords-based socially tenuous group (KTG) queries. Luo et al. proposed the MCR-Tree [26], an efficient R-tree-based index for multi-dimensional core search. Zhong et al. [27] presented a unified algorithm framework for temporal $(k, \mathcal{X})$-core queries, extending temporal k-core queries to handle user-defined metrics with time-insensitive or time-monotonic properties via a two-phase approach. Yang et al. [28] introduced BCList and BCList++ for (p, q)-biclique counting and enumeration in large sparse bipartite graphs. Two works relevant to our study are [2], [3], which proposed the radius-bounded k-core searches problem.

Refined query exploration for why-not questions: Why-not question was first formalized by Chapman and Jagadish [29], which explained why-not question by identifying the "culprit" operation that leads to the exclusion of expected tuples. A new explanation framework is proposed by [5], which is to explore a refined query to ensure that the refined result contains both the original query result and the missing result. Similar techniques are widely used by [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44] to solve the why-not problemsAnd Chen et al. [6], [45], [46], [47] addressed missing values on spatial keyword top-k queries with regard to different aspects. Furthermore, After that, Song et al. [48], [49] conducted why-not questions on subgraph queries in multi-attributed graphs. The why-not questions of top-k spatial keyword queries are investigated by [50]. Additionally, [51] took a different approach from the aspect of causality, the problem of missing tuples and redundant tuples was studied. [52], [53] explored refined queries for the missing answers over nested data.

VIII. Conclusion

In this paper, we address the *Exploring Optimal Parameters* problem to identify optimal parameters for *expected* (EOPE) or *unexpected* (EOPU) results in RB-k-core queries. We first introduce a novel explanation framework. For EOPE, we propose two baseline exploration algorithms, PriorityR and HybridR, based on effective bounds of the refined r' parameter and a dichotomous strategy. Subsequently, by further tightening the bounds of refined k' and r', we develop two more efficient exploration algorithms, PriorityK and HybridK. Additionally, we design an effective index structure, HCR-Tree, which combines hierarchical coreness and R-Tree to enhance the performance of the proposed exploration algorithms. For EOPU, we propose a basic solution (BS) using the three-vertices method, and then develop an effective algorithm, SA, by incorporating pruning and termination strategies. Extensive experiments validate the efficiency and effectiveness of the proposed index and algorithms.

References

[1] J. Wu, Y. Zhang, Y. Li, Y. Zou, R. Li, and Z. Zhang, "SSTP: Social and spatial-temporal aware next point-of-interest recommendation," *Data Sci. Eng.*, vol. 8, no. 4, pp. 329–343, 2023.

[2] K. Wang, X. Cao, X. M. Lin, W. J. Zhang, and L. Qin, "Efficient computing of radius-bounded k-cores," in *Proc. Int. Conf. Data Eng.*, 2018, pp. 233–244.

[3] K. Wang, S. T. Wang, X. Cao, and L. Qin, "Efficient radius-bounded community search in geo-social networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 9, pp. 4186–4200, Sep., 2022.

[4] C. Zong et al., "Exploring optimal parameters for expected results on radius-bounded k-core queries," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 2312–2324.

[5] Q. T. Tran and C. Y. Chan, "How to conquer why-not questions," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2010, pp. 15–26.

[6] L. Chen, X. Lin, H. B. Hu, C. S. Jensen, and J. L. Xu, "Answering why-not questions on spatial keyword top-k queries," in *Proc. Int. Conf. Data Eng.*, 2015, pp. 279–290.

[7] C. Y. Zong et al., "Answering why-not questions on structural graph clustering," in *Proc. 23rd Int. Conf. Database Syst. Adv. Appl.*, 2018, pp. 255–271.

[8] C. Y. Zong, Z. F. Dong, X. C. Yang, B. Wang, T. Qiu, and H. J. Zhu, "Efficiently answering why-not questions on radius-bounded k-core searches," in *Proc. 23rd Int. Conf. Database Syst. Adv. Appl.*, 2023, pp. 93–109.

[9] M. S. Islam, C. F. Liu, and J. X. Li, "Efficient answering of why-not questions in similar graph matching," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 10, pp. 2672–2686, Oct., 2015.

[10] Q. Song, M. H. Namaki, P. Lin, and Y. Wu, "Answering why-questions for subgraph queries," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4636–4649, Oct., 2022.

[11] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1984, pp. 47–57.

[12] V. Batagelj and M. Zaversnik, "An o(m) algorithm for cores decomposition of networks," 2003, *arXiv:cs/0310049*.

[13] Y. Fang, R. Cheng, X. Li, S. Luo, and J. Hu, "Effective community search over large spatial graphs," *Proc. VLDB Endowment*, vol. 10, no. 6, pp. 709–720, 2017.

[14] E. Cho, S. A. Myers, and J. Leskovec, "Friendship and mobility: User movement in location-based social networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2011, pp. 1082–1090.

[15] M. Sarwat, J. J. Levandoski, A. Eldawy, and M. F. Mokbel, "LARS*: An efficient and scalable location-aware recommender system," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 6, pp. 1384–1399, Jun., 2014.

[16] M. Sozio and A. Gionis, "The community-search problem and how to plan a successful cocktail party," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2010, pp. 939–948.

[17] J. Kim, T. Guo, K. Y. Feng, G. Cong, A. Khan, and F. M. Choudhury, "Densely connected user community and location cluster search in location-based social networks," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2199–2209.

[18] L. Chen, C. F. Liu, R. Zhou, J. X. Li, X. C. Yang, and B. Wang, "Maximum co-located community search in large scale social networks," *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1233–1246, 2018.

[19] Y. Chen, J. Xu, and M. Z. Xu, "Finding community structure in spatially constrained complex networks," *Int. J. Geographical Inf. Sci.*, vol. 29, no. 6, pp. 889–911, 2015.

[20] Q. Zhu, H. Hu, C. Xu, J. Xu, and W. C. Lee, "Geo-social group queries with minimum acquaintance constraints," in *Proc. VLDB Endowment*, vol. 26, pp. 709–727, 2017.

[21] X. Y. Miao, Y. Liu, L. Chen, Y. J. Gao, and J. W. Yin, "Reliable community search on uncertain graphs," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 1166–1179.

[22] L. Dhulipala, Q. C. Liu, S. Raskhodnikova, J. Shi, J. Shun, and S. Yu, "Differential privacy from locally adjustable graph algorithms: K-core decomposition, low out-degree ordering, and densest subgraphs," in *Proc. IEEE 63rd Annu. Symp. Foundations Comput. Sci.*, 2022, pp. 754–765.

[23] X. F. Li, R. Zhou, L. Chen, C. F. Liu, Q. He, and Y. Yang, "One set to cover all maximal cliques approximately," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2022, pp. 2006–2019.

[24] H. Zhu et al., "Continuous GEO-social group monitoring over moving users," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 312–324.

[25] H. Zhu et al., "Keyword-based socially tenuous group queries," in *Proc. Int. Conf. Data Eng.*, 2023, pp. 965–977.

[26] C. Luo, Y. Zhu, Q. Liu, Y. Gao, L. Chen, and J. Xu, "MCR-tree: An efficient index for multi-dimensional core search," *Proc. ACM Manage. Data*, vol. 2, no. 3, pp. 1–25, 2024.

[27] M. Zhong, J. Yang, Y. Zhu, T. Qian, M. Liu, and J. X. Yu, "A unified and scalable algorithm framework of user-defined temporal (K, X)-core query," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 2831–2845, Jul., 2024.

[28] J. Yang, Y. Peng, D. Ouyang, W. Zhang, X. Lin, and X. Zhao, "(P, Q)-biclique counting and enumeration for large sparse bipartite graphs," *VLDB J.*, vol. 32, no. 5, pp. 1137–1161, 2023.

[29] A. Champan and H. V. Jagadish, "Why not '?'," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2009, pp. 523–534.

[30] Z. Chen, P. Manolios, and M. Riedewald, "Why not yet: Fixing a top-k ranking that is not fair to individuals," *Proc. VLDB Endowment*, vol. 16, no. 9, pp. 2377–2390, 2023.

[31] Y. Li, W. Zhang, Y. Gao, Q. Li, L. Shu, and C. Luo, "DAPC: Answering why-not questions on top-k direction-aware ASK queries in polar coordinates," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 5, pp. 4932–4947, May, 2023.

[32] Z. He and E. Lo, "Answering why-not questions on top-k queries," in *Proc. Int. Conf. Data Eng.*, 2012, pp. 750–761.

[33] Z. He and E. Lo, "Answering why-not questions on top-k queries," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 6, pp. 1300–1315, Jun., 2014.

[34] L. Chen, Y. Li, J. Xu, and C. S. Jensen, "Direction-aware why-not spatial keyword top-k queries," in *Proc. 33rd IEEE Int. Conf. Data Eng.*, San Diego, CA, USA, 2017, pp. 107–110.

[35] L. Chen, Y. Gao, K. Wang, C. S. Jensen, and G. Chen, "Answering why-not questions on metric probabilistic range queries," in *Proc. 32nd IEEE Int. Conf. Data Eng.*, Helsinki, Finland, 2016, pp. 767–778.

[36] Q. Liu, Y. Gao, L. Zhou, and G. Chen, "IS2R: A system for refining reverse top-k queries," in *Proc. 33rd IEEE Int. Conf. Data Eng.*, San Diego, CA, USA, 2017, pp. 1371–1372.

[37] B. Li, H. Duan, Y. Liu, L. Zhang, W. Cui, and J. T. Zhou, "STADe: Sensory temporal action detection via temporal-spectral representation learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 9, pp. 8117–8133, Sep., 2025.

[38] W. Xu, Z. He, E. Lo, and C.-Y. Chow, "Explaining missing answers to top-k SQL queries," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 8, pp. 2071–2085, Aug., 2016.

[39] Y. J. Gao, Q. Liu, G. Chen, B. H. Zheng, and L. L. Zhou, "Answering why-not questions on reverse top-k queries," in *Proc. Int. Conf. Very Large Data Bases*, 2015, pp. 738–749.

[40] Q. Liu, Y. J. Gao, G. Chen, B. H. Zheng, and L. L. Zhou, "Answering why-not and why questions on reverse top-k queries," *VLDB J.*, vol. 25, no. 6, pp. 867–892, 2016.

[41] J. W. Zhao, Y. J. Gao, G. Chen, and R. Chen, "Why-not questions on top-k GEO-social keyword queries in road networks," in *Proc. Int. Conf. Data Eng.*, 2018, pp. 965–976.

[42] M. S. Islam, R. Zhou, and C. F. Liu, "On answering why-not questions in reverse skyline queries," in *Proc. Int. Conf. Data Eng.*, 2013, pp. 973–984.

[43] X. Y. Miao, Y. J. Gao, S. Guo, and G. Chen, "On efficiently answering why-not range-based skyline queries in road networks," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1697–1711, Sep., 2018.

[44] M. H. Namaki, Q. Song, Y. H. Wu, and S. Q. Yang, "Answering why-questions by exemplars in attributed graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2019, pp. 1481–1498.

[45] L. Chen, J. L. Xu, X. Lin, C. S. Jensen, and H. B. Hu, "Answering why-not spatial keyword top-k queries via keyword adaption," in *Proc. Int. Conf. Data Eng.*, 2016, pp. 697–708.

[46] L. Chen, J. L. Xu, C. S. Jensen, and Y. F. Li, "YASK: A why-not question answering engine for spatial keyword query services," *Proc. VLDB Endowment*, vol. 9, no. 13, pp. 1501–5104, 2015.

[47] L. Chen, Y. Li, J. Xu, and C. S. Jensen, "Towards why-not spatial keyword top-k queries: A direction-aware approach," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 4, pp. 796–809, Apr., 2018.

[48] Q. Song, M. H. Namaki, and Y. H. Wu, "Answering why-questions for subgraph queries in multi-attributed graphs," in *Proc. Int. Conf. Data Eng.*, 2019, pp. 40–51.

[49] Q. Song, P. Lin, H. Ma, and Y. Wu, "Explaining missing data in graphs: A constraint-based approach," in *Proc. 37th IEEE Int. Conf. Data Eng.*, Chania, Greece, 2021, pp. 1476–1487.

[50] B. Zheng et al., "Answering why-not group spatial keyword queries," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 1, pp. 26–39, Jan., 2020.

[51] S. Roy and D. Suciu, "A formal approach to finding explanations for database queries," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1579–1590.

[52] R. Diestelkämper, S. Lee, M. Herschel, and B. Glavic, "To not miss the forest for the trees—A holistic approach for explaining missing answers over nested data," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 405–417.

[53] R. Diestelkämper, S. Lee, B. Glavic, and M. Herschel, "Debugging missing answers for spark queries over nested data with breadcrumb," *Proc. VLDB Endowment*, vol. 14, no. 12, pp. 2731–2734, 2021.

Zefang Dong received the MSc degree in computer science from Shenyang Aerospace University, China, in 2024. He is currently working toward the EnD degree in computer science and technology with Northeastern University, China. His research interests include data provenance, query processing, and optimization.

Chuanyu Zong received the PhD degree in computer science from Northeastern University, China, in 2017. He is currently an associate professor with the Department of Computer Engineering, Shenyang Aerospace University, China. His research interests include data quality management, data trace ability, query processing and optimization, and graph structured data processing.

Xiaochun Yang (Senior Member, IEEE) received the PhD degree in computer science from Northeastern University, China, in 2001. She is currently a professor with the Department of Computer Science, Northeastern University. Her research interests include Big Data management and knowledge engineering, data quality, and data privacy.

Bin Wang received the PhD degree in computer science from Northeastern University, China, in 2008. He is currently a professor with the Department of Computer Engineering, Northeastern University, China. His research interests include query processing and optimization, text and time-series data processing and analysis, machine learning, and recommendation algorithms.

Boce Chu received the PhD degree in signal and information processing from Beihang University, China, in 2024. He is currently a senior engineer with the Hebei University of Economics and Business. His research interests include algorithms for machine learning, data mining, and image processing.

Wang Yaqi is currently working toward the PhD degree with the School of Computer Science and Engineering, Northeastern University. Her research interests include learned indices and query processing.

Huaijie Zhu received the BSc degree from Information Science Department, Kunming University of Science and Technology, the MSc degree from Computer Science Department, Northeastern University, and the PhD degree in computer science from Northeastern University, China, in 2018. He is currently an associate professor with Sun Yat-sen University. His research interests include spatial database and data privacy.